5-Bit Carry Look-ahead Adder

VLSI Course Project Report

Submitted by:

A.Pranav
2024102038
pranav.aravind@students.iiit.ac.in

Department of Electronics and Communication Engineering

International Institute of Information Technology
Hyderabad, India

Contents

Abstract

This report details the design and implementation of a 5-Bit Carry Look-ahead Adder (CLA). The CLA improves speed by reducing the propagation delay associated with standard ripple carry adders.

I Introduction

Adders are widely used sub-circuits in almost all major circuits. To add two 'n'-bit numbers, a primitive way is to chain together 'n' full-adders such that the 'carry-out' of the previous stage acts as the 'carry-in' of the next stage. However, a major disadvantage is the speed of the circuit. In order for the 'i'th sum-bit to be computed, all the previous carry bits must be rippled through the circuit. The rippling of carry is a huge bottleneck. A clever way to solve this issue is to pre-compute the carry-bits from the input bits itself. This is called the 'Carry-Lookahead' logic. By pre-computing the carry bits, we do not need to wait for the carry to be rippled through the entire circuit, which results in a much faster adder. The following report details the implementation of a 5-bit Carry Look-ahead Adder.

II Proposed Circuit

II.A Circuit Overview

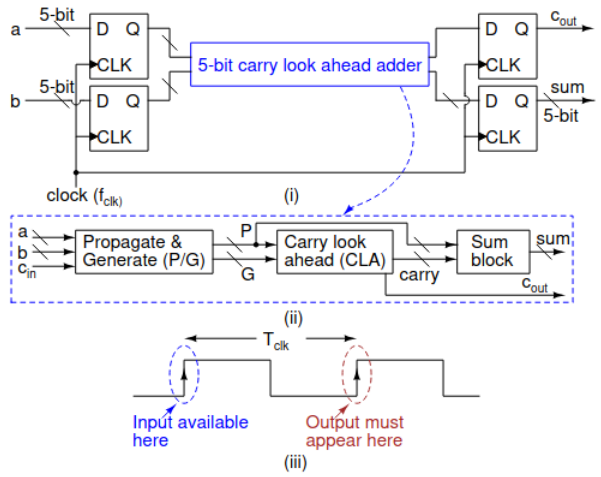

Figure 1: Circuit Overview

The circuit uses input flip flops for sequencing. The propagate and generate signals are computed in the (P&G) block. The carry is computed in the carry block and the sum is computed in the sum block. The output sum and carry bits are passed through flip flops for synchronization.

II.B Design Equations

Before defining the recursive carry equations, we must establish the logic for the Propagate (P_i) and Generate (G_i) signals. While the strict definition of Propagate is $P_i = A_i \oplus B_i$, it is permissible to use the OR operation ($A_i + B_i$) for carry generation logic.

Table 1 compares the truth tables for XOR and OR.

Table 1: Truth Table Comparison

A_i	B_i	$A_i \oplus B_i$	$A_i + B_i$
0	0	0	0
0	1	1	1
1	0	1	1
1	1	0	1

The discrepancy occurs only when $A_i = 1$ and $B_i = 1$. In this specific case, the Generate signal ($G_i = A_i \cdot B_i$) will definitely be 1.

Consider the carry recurrence relation:

$$C_{i+1} = G_i + (P_i \cdot C_i) \tag{1}$$

If $A_i = 1$ and $B_i = 1$, then $G_i = 1$. Consequently, the carry out C_{i+1} becomes 1 regardless of the value of the term $(P_i \cdot C_i)$. Therefore, the Propagate term P_i can be implemented using an OR gate without affecting the correctness of the carry output,simplifying the transistor layout.

Based on this, the equations for the 5-bit adder are defined as follows:

$$P_i = A_i + B_i \quad \text{(using OR simplification)} \tag{2}$$
$$G_i = A_i \cdot B_i \tag{3}$$

Using the recursive definition, the expanded equations for bits 1 through 5 are:

$$C_1 = G_0 + P_0 C_0 \tag{4}$$
$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0 \tag{5}$$
$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \tag{6}$$
$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0 \tag{7}$$

The final carry out (C_5) for the 5th bit is:

$$C_5 = G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1 + P_4 P_3 P_2 P_1 G_0 + P_4 P_3 P_2 P_1 P_0 C_0 \tag{8}$$

Finally, the Sum bits (S_i) are calculated using the Exclusive-OR operation between the Propagate signal and the incoming carry. The relation is given by:

$$S_i = P_i \oplus C_i \tag{9}$$

III Optimization

From the above functions, we can observe that we require upto 6 input AND and OR gates to realize the outputs. 6 input gates require a large number of transistors. They have a large logical effort and parasitic delay. Hence the layout will have a large area and the circuit will be slow. Hence to optimize for speed and area, we use a novel implementation proposed by Miao and Li [1]. In this implementation, we

use De-Morgan's Laws to rewrite the functions such that we require a maximum of only two-input gates. The two input gates will have much lesser logical effort and parasitic delay and hence the circuit will be much faster.

III.A Derivation of Carry Equations

In this design, we optimize the carry logic by converting the equations into a form suitable for CMOS implementation, utilizing complemented variables.

We are given the initial condition $C_0 = 0$.

Carry 1:

$$C_1 = G_0 + P_0C_0 = G_0 \tag{10}$$

Carry 2:

$$\begin{aligned}
C_2 &= G_1 + P_1G_0 \\
C_2 &= \overline{\overline{G_1 + P_1G_0}} \\
C_2 &= \overline{\overline{G_1} \cdot \overline{P_1G_0}} \\
C_2 &= \overline{\overline{G_1} \cdot (\overline{P_1} + \overline{G_0})}
\end{aligned}$$

Carry 3:

$$\begin{aligned}
C_3 &= G_2 + P_2C_2 \\
C_3 &= \overline{\overline{G_2 + P_2G_1 + P_2P_1G_0}} \\
C_3 &= \overline{\overline{G_2} \cdot (\overline{P_2} + \overline{G_1}) \cdot (\overline{P_2} + \overline{P_1} + \overline{G_0})} \\
C_3 &= \overline{\left[\overline{G_2} \cdot (\overline{P_2} + \overline{G_1})\right] \cdot \left[(\overline{P_1} + \overline{P_2}) + \overline{G_0}\right]} \\
C_3 &= \overline{\left[\overline{G_2} \cdot (\overline{P_2} + \overline{G_1})\right]} + \overline{\left[(\overline{P_1} + \overline{P_2}) + \overline{G_0}\right]} \\
C_3 &= \overline{\overline{G_2} \cdot (\overline{P_2} + \overline{G_1})} + \left(\overline{(\overline{P_1} + \overline{P_2})} \cdot \overline{\overline{G_0}}\right) \\
C_3 &= \overline{\overline{G_2} \cdot (\overline{P_2} + \overline{G_1})} + \overline{(\overline{P_1} + \overline{P_2})} \cdot G_0
\end{aligned}$$

Carry 4:

$$\begin{aligned}
C_4 &= G_3 + P_3C_3 \\
C_4 &= G_3 + P_3(G_2 + P_2G_1 + P_2P_1G_0) \\
C_4 &= G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 \\
C_4 &= \overline{\overline{G_3 + P_3G_2 + P_3P_2(G_1 + P_1G_0)}} \\
C_4 &= \overline{\overline{\left[\overline{G_3} \cdot (\overline{P_3} + \overline{G_2})\right]} \cdot \overline{\left[P_3P_2 + \overline{G_1 + P_1G_0}\right]}} \\
C_4 &= \overline{\overline{\left[\overline{G_3} \cdot (\overline{P_3} + \overline{G_2})\right]} \cdot \left[(\overline{P_3} + \overline{P_2}) + \overline{G_1} \cdot (\overline{P_1} + \overline{G_0})\right]} \\
C_4 &= \overline{\overline{G_3} \cdot (\overline{P_3} + \overline{G_2})} + \left(\overline{\overline{P_3} + \overline{P_2}}\right) \cdot \left(\overline{\overline{G_1} \cdot (\overline{P_1} + \overline{G_0})}\right)
\end{aligned}$$

Carry 5:

$$\begin{aligned}
C_5 &= G_4 + P_4C_4 \\
C_5 &= G_4 + P_4(G_3 + P_3G_2 + P_3P_2C_2) \\
C_5 &= (G_4 + P_4G_3) + P_4P_3(G_2 + P_2C_2) \\
C_5 &= \overline{\overline{(G_4 + P_4G_3) + P_4P_3C_3}} \\
C_5 &= \overline{\overline{G_4 + P_4G_3} \cdot \overline{P_4P_3C_3}} \\
C_5 &= \overline{\overline{\left[G_4 \cdot (\overline{P_4} + \overline{G_3})\right] \cdot \left[P_4P_3 + \overline{C_3}\right]}} \\
C_5 &= \overline{\overline{G_4} \cdot (\overline{P_4} + \overline{G_3})} + \overline{\overline{P_4} + \overline{P_3}} \cdot C_3 \\
C_5 &= \overline{\overline{G_4} \cdot (\overline{P_4} + \overline{G_3})} + \overline{(\overline{P_4} + \overline{P_3})} \cdot \left[\overline{\overline{G_2} \cdot (\overline{P_2} + \overline{G_1})} + \overline{(\overline{P_1} + \overline{P_2})} \cdot G_0\right]
\end{aligned}$$

Implementing the above equations, we get the optimized circuit.From the modified equations, we observe that there are only two input gates used. This novelty massively boosts the speed of the circuit.

IV Flip Flop Design

The flip flops are used as sequencing elements to ensure the inputs and outputs are synchronized with the clock. We use the True-Single Phase Clock (TSPC) design to implement the flip flop. This approach eliminates the need to generate the clock and its complement. Generating the complement of the clock often leads to skew and causes undesirable overlap between the clock phases. We use the optimized 11-transistor implementation of the TSPC Flip Flop [2].

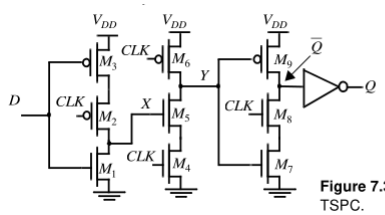

Figure 7.33 *Positive edge-triggered* register TSPC.

Figure 2: TSPC Flip-Flop

IV.A Sizing the Flip Flop

Transistor sizing is critical for achieving correct functionality in the TSPC register. With improper sizing, glitches may occur at the output due to a race condition when the clock transitions from low to high. Consider the case where D is low and $Q = 1$ ($Q = 0$). While CLK is low, Y is precharged high turning on M_7. When CLK transitions from low to high, nodes Y and Q start to discharge simultaneously (through M_4-M_5 and M_7-M_8, respectively). Once Y is sufficiently low, the trend on Q is reversed and the node is pulled high anew through M_9.

The problem can be corrected by resizing the relative strengths of the pull-down paths through M_4-M_5 and M_7-M_8, so that Y discharges much faster than Q. This is accomplished by reducing the strength of the M_7-M_8 pulldown path, and by speeding up the M_4-M_5 pulldown path.

Table 2 details the specific transistor sizes chosen to satisfy these constraints, where the base width W is defined as the width of an NMOS in a minimum-sized inverter ($W = 10\lambda$).

Table 2: Transistor Sizing for TSPC Flip Flop ($W = 10\lambda$)

Transistor	Relative Width	Width in λ
M_1	W	10λ
M_2	$4W$	40λ
M_3	$4W$	40λ
M_4	$4W$	40λ
M_5	$4W$	40λ
M_6	$2W$	20λ
M_7	$2W$	20λ
M_8	$2W$	20λ
M_9	$2W$	20λ

IV.B Flip Flop Layout

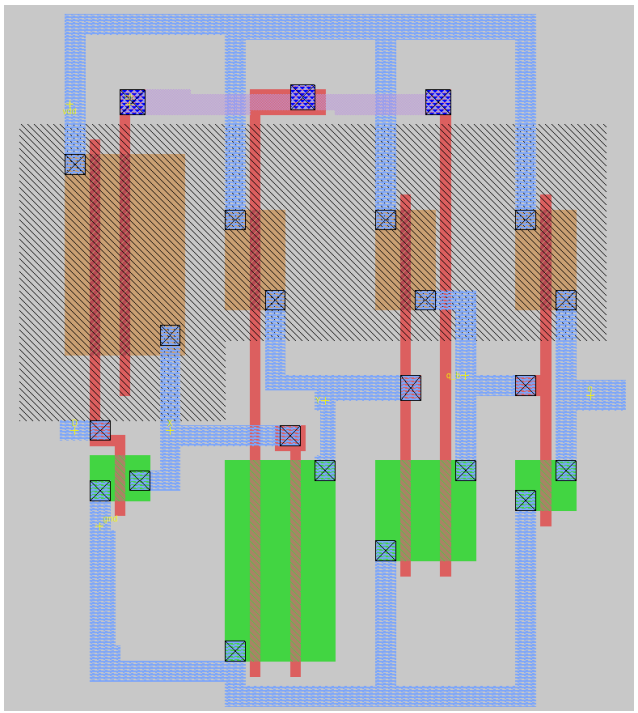

Figure 3: TSPC Flip Flop Layout

IV.C Flip Flop Simulations

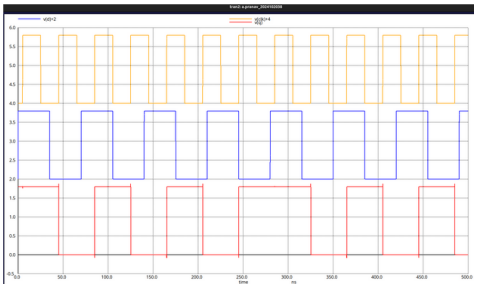

Figure 4: Prelayout Simulation

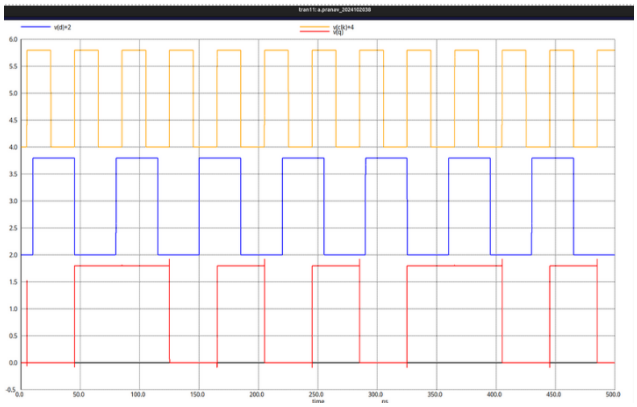

Figure 5: Post layout simulation

IV.D Flip Flop Characterization

IV.D.1 Setup Time

The setup time for the flip flop is defined as the minimum time before the rising edge of the clock for which the input data is to be held constant. For our TSPC design, the time it takes for the value to go from D to X, i.e, one inverter delay is the setup time.

Pre Layout Setup Time=70.87ps

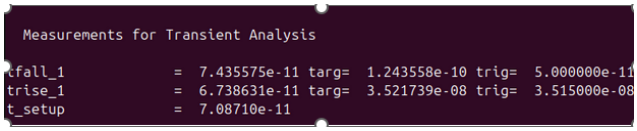

Figure 6: Pre Layout Setup Time

Post Layout Setup Time=48.6ps

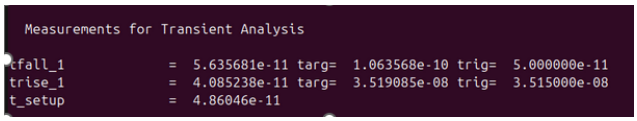

Figure 7: Post Layout Setup Time

IV.D.2 Hold Time

Hold Time of the flip flop is defined as the minimum time after the rising edge of the clock when the input data is to be held constant. For the TSPC design, the hold time is the time taken for data to propogate between X and Y . Pre Layout Hold Time=39.4 ps

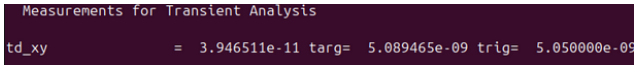

Figure 8: Pre Layout Hold Time

Post Layout Hold Time=26.8 ps

```
Measurements for Transient Analysis

td_xy              =  2.686400e-11 targ=  5.076864e-09 trig=  5.050000e-09
```

Figure 9: Post Layout Hold Time

IV.D.3 Clock to Q delay

Clock to Q delay t_{CQ} is defined as the time it takes for Q to sample D after the clock reaches the rising edge.
Prelayout t_{CQ} = 91 ps

```
Measurements for Transient Analysis

tfall_1            =  1.198164e-10 targ=  5.169816e-09 trig=  5.050000e-09
trise_1            =  6.409528e-11 targ=  4.511410e-08 trig=  4.505000e-08
t_cq               =  9.19559e-11
```

Figure 10: Prelayout t_{CQ}

Postlayout t_{CQ} = 58 ps

```
Measurements for Transient Analysis

tfall_1            =  3.785403e-11 targ=  5.087854e-09 trig=  5.050000e-09
trise_1            =  7.986604e-11 targ=  4.512987e-08 trig=  4.505000e-08
t_cq               =  5.88600e-11
```

Figure 11: Post Layout t_{CQ}

V Description of the unique gates used

V.A XOR Gate

I have used a Pass Transistor Logic (PTL) implementation of the XOR gate. This is because it is much easier and requires much less transistors than the CMOS implementation. The CMOS implementation requires around 12 transistors while the PTL implementation requires only 8 transistors. We have also used an inverter at the end to restore the degraded logic, as NMOS passes a weak HIGH.

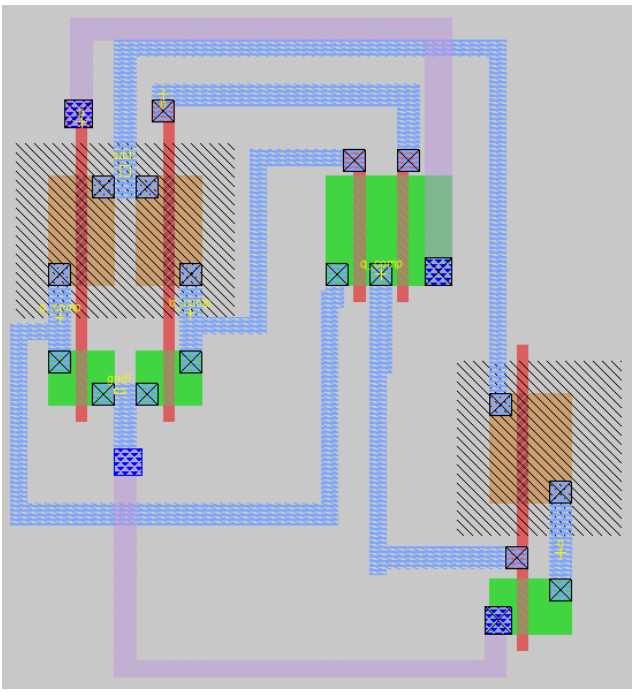

Figure 12: Layout of XOR

V.A.1 Sizing

The inverters at the input to generate the complements are minimum sized. PMOS width=20λ and NMOS width=10λ. The NMOS transistors in the PTL structure are sized as 20λ each to speed up the charging and discharging.

V.A.2 Simulation

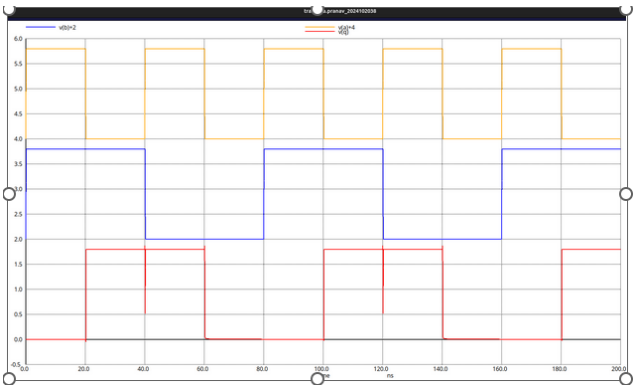

Figure 13: Prelayout XOR

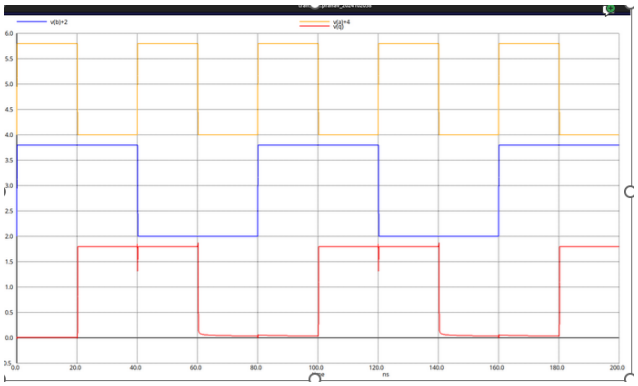

Figure 14: Post Layout XOR

V.B Other Gates

All the other gates are implemented using static CMOS logic. The sizing is done so that the worst case pull up or pull down resistance matches the pull up or pull down resistance of a minimum sized inverter. This is done to minimize the delay.

Table 3: Gate Sizing Dimensions

Gate	NMOS Width	PMOS Width
Inverter	10λ	20λ
NAND	20λ	20λ
NOR	10λ	40λ
AND	20λ	20λ
OR	10λ	40λ

V.B.1 Gate Layouts

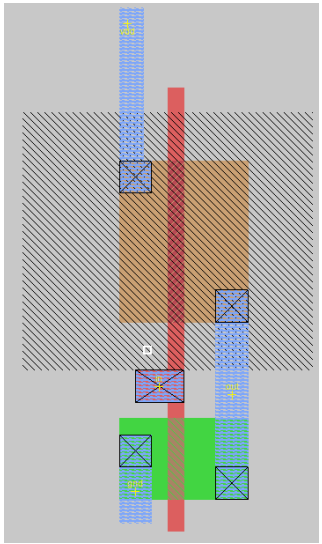

Figure 15: Inverter

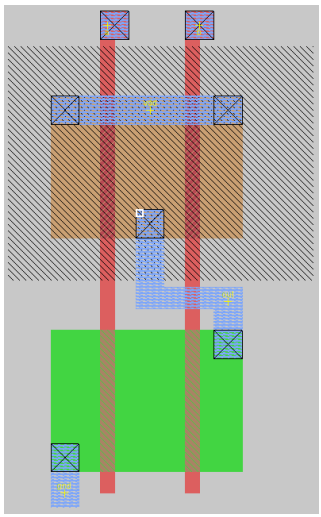

Figure 16: NAND

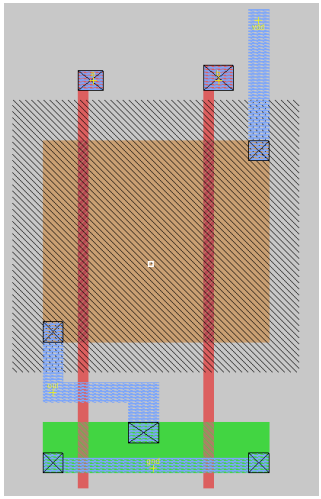

Figure 17: NOR

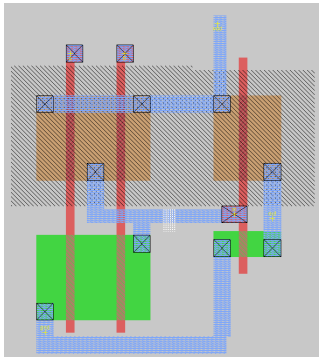

Figure 18: AND

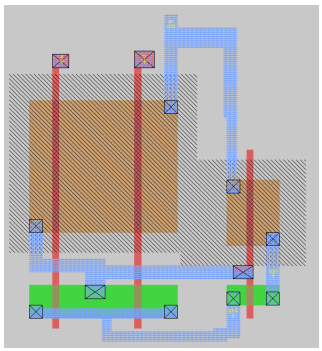

Figure 19: OR

V.B.2 Pre Layout Simulations

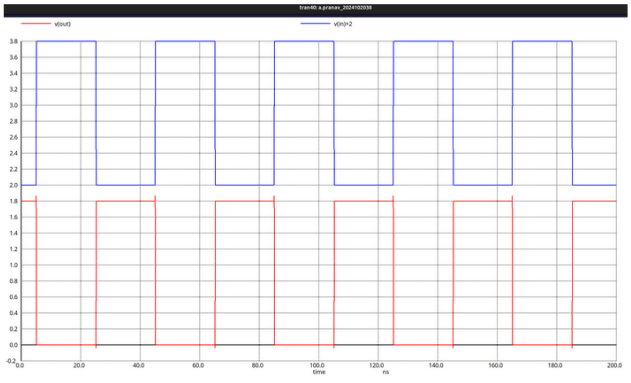

Figure 20: Inverter

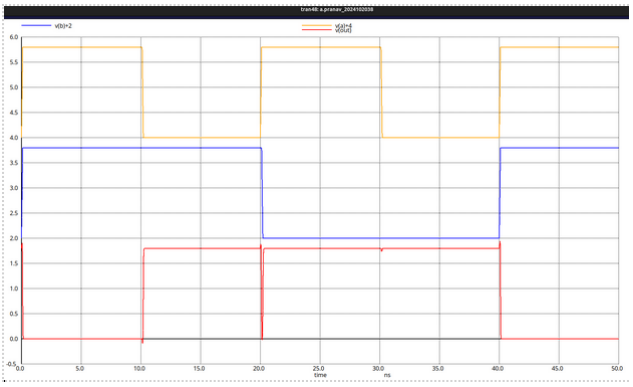

Figure 21: NAND

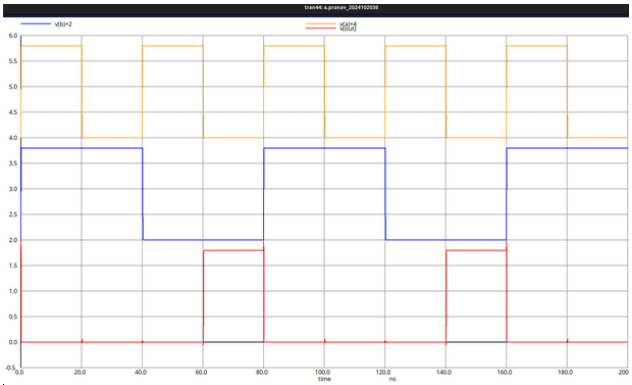

Figure 22: NOR

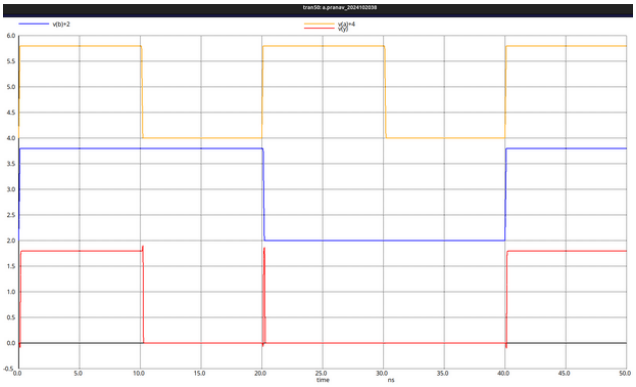

Figure 23: AND

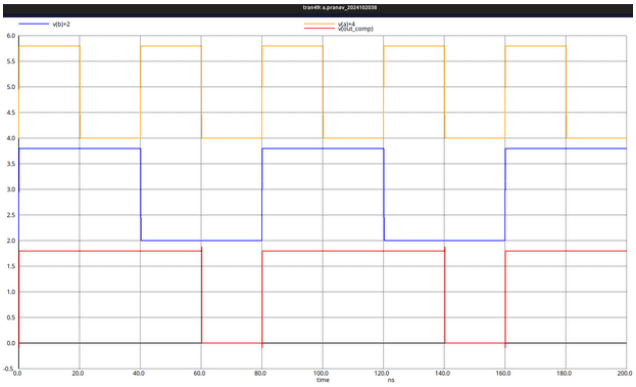

Figure 24: OR

V.B.3 Post Layout Simulations

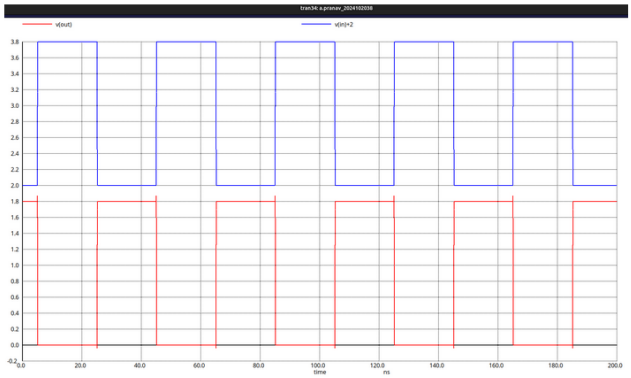

Figure 25: Inverter

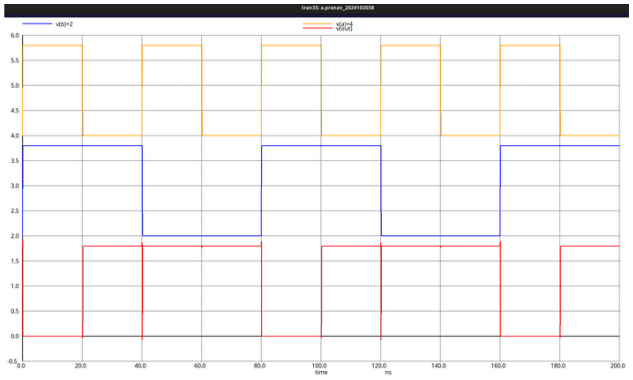

Figure 26: NAND

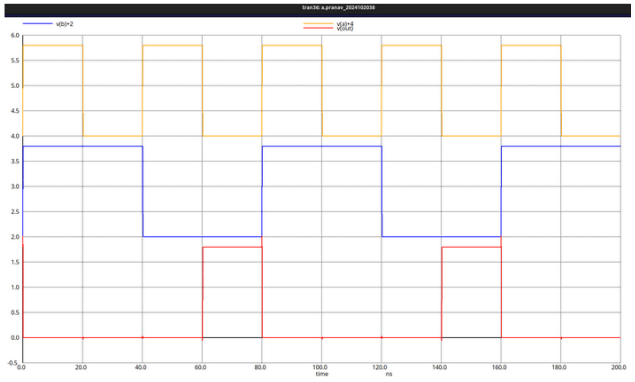

Figure 27: NOR

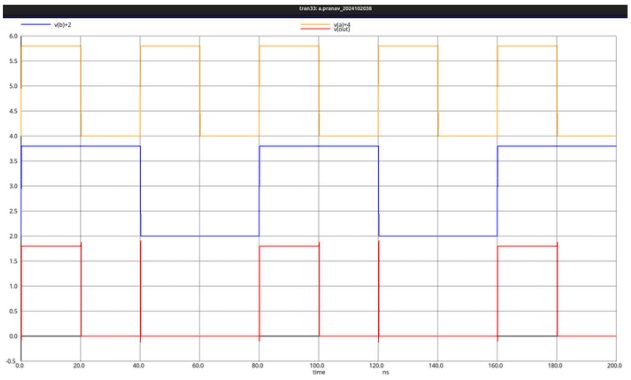

Figure 28: AND

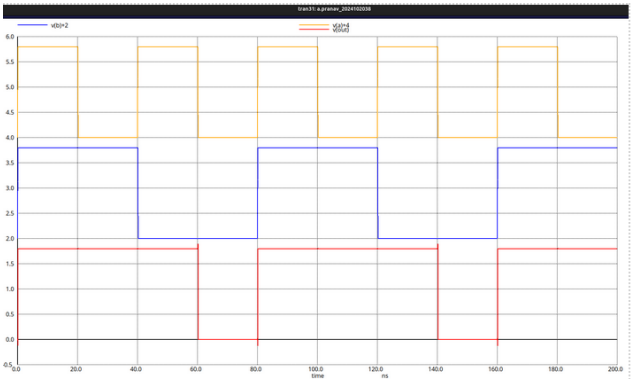

Figure 29: OR

VI Stick Diagrams

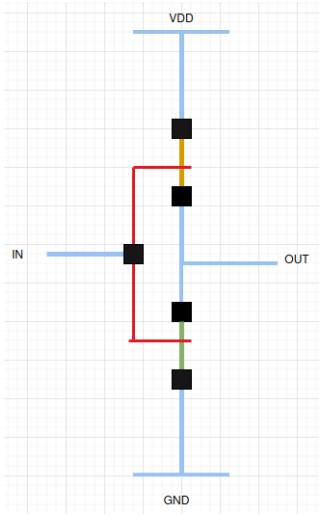

Figure 30: Inverter

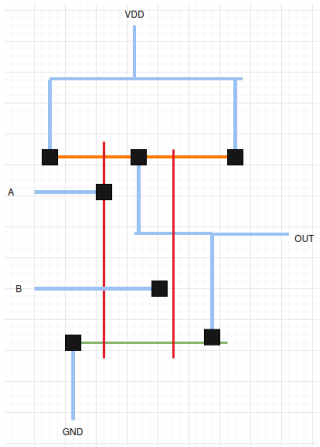

Figure 31: NAND

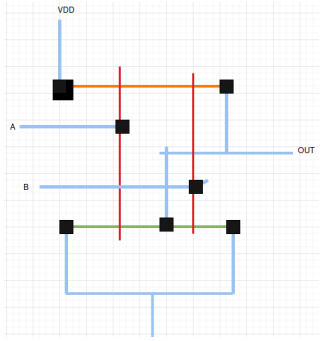

Figure 32: NOR

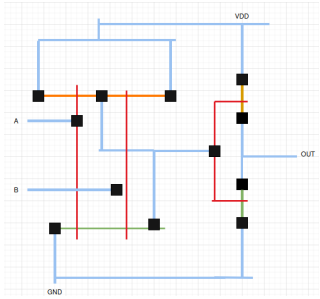

Figure 33: AND

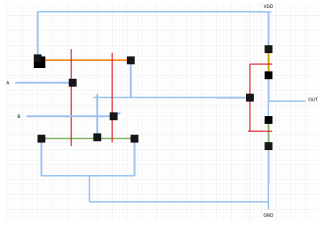

Figure 34: OR

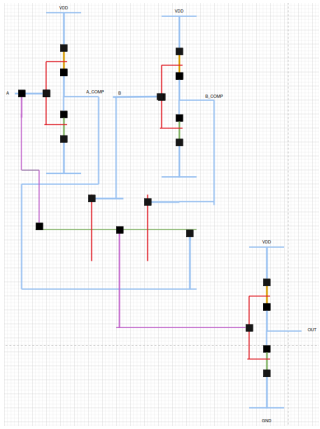

Figure 35: XOR

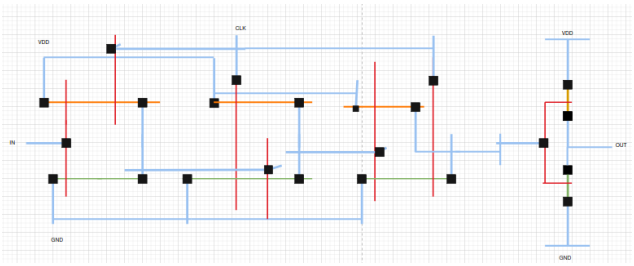

Figure 36: TSPC Flip Flop

VII PG Block

The PG block consists of NAND and NOR gates to generate the $G_i s$ and $P_i s$ respectively.

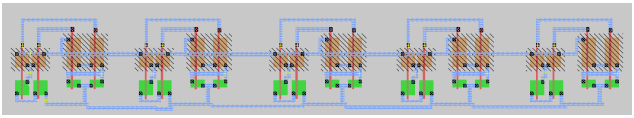

Figure 37: Layout of PG Block

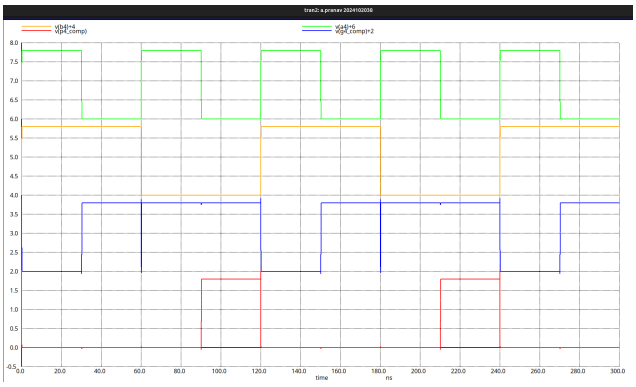

Figure 38: Simulation of PG Block

VIII Carry Block

The carry block consists of the logic to implement the boolean functions as discussed in the proposed logic in order to generate the carry bits.

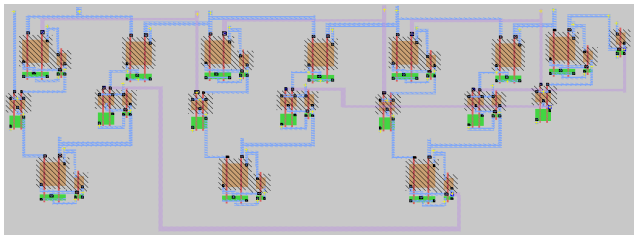

Figure 39: Layout of Carry Block

I test the carry block for all possible combinations of inputs.

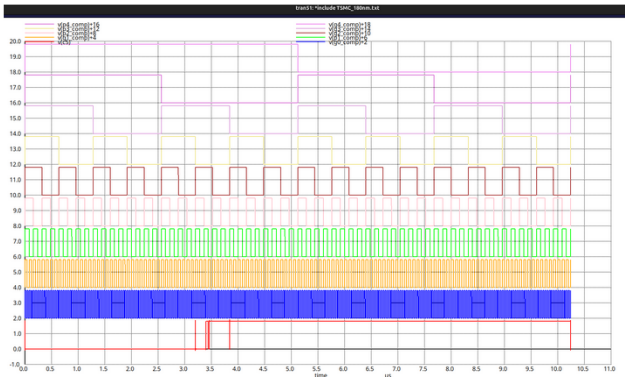

Figure 40: Result of Carry Block

IX Final Circuit

The input flip flops, the PG Block, the Carry Block, the Sum logic and the output flip flops are integrated to make the final circuit. At the output of each bit, we add a minimum sized inverter as load.

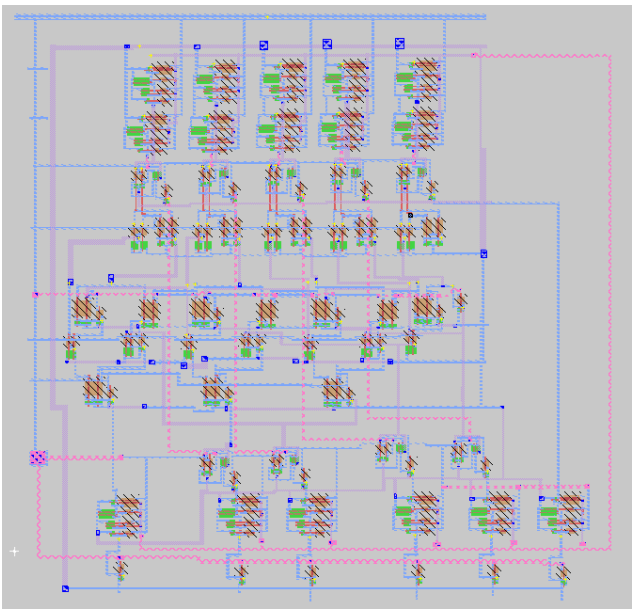

Figure 41: Layout of Final Circuit

I test the final circuit for a particular input test case.
A=11101 (29)
B=10011 (19)

INPUT	Value
a0	1
a1	0
a2	1
a3	1
a4	1
b0	1
b1	1
b2	0
b3	0
b4	1

Table 4: Input logic values

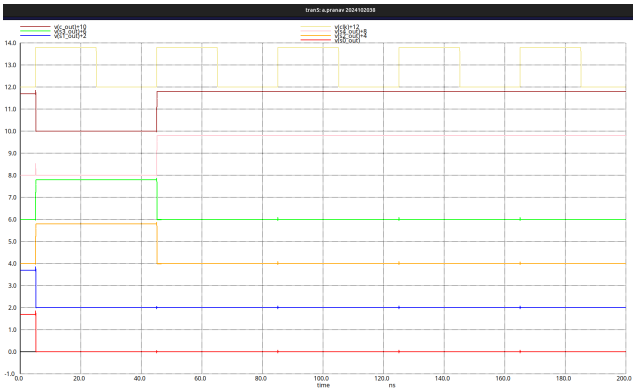

Figure 42: Result of Final Circuit

Note that the output settles only after the second clock edge due to the two layers of flip flops. The output is 11000

(48) which is the correct answer. Hence the circuit works as expected.

I wrote the Pre Layout NGSPICE for the final circuit and simulated for the same case.

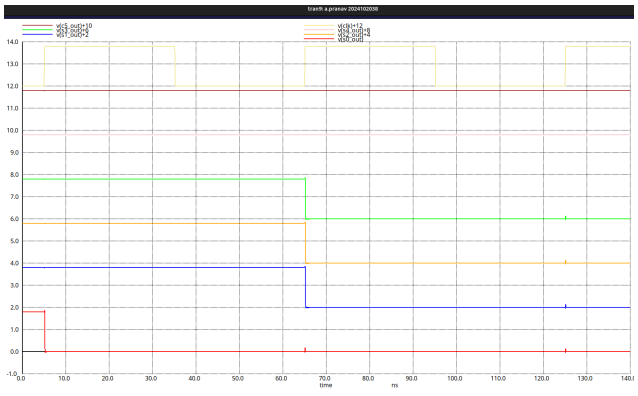

Figure 43: Prelayout simulation

We get the same result.

IX.A Horizontal and Vertical Pitch

I calculate it by finding the box size of the entire circuit.

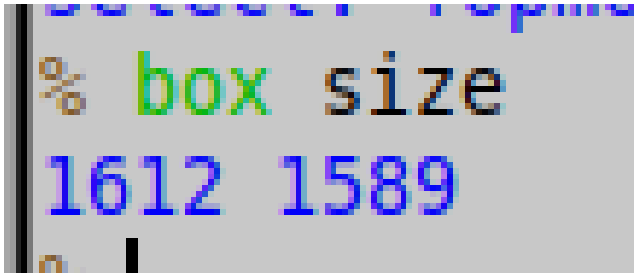

Figure 44: Pitch

Horizontal Pitch = $1612\lambda = 145.08\mu$m
Vertical Pitch = $1598\lambda = 143.01\mu$m

IX.B Worst Case Delay

IX.B.1 Pre layout

We get the worst case delay in the following case:
A=11011
B=10110

We get the worst case delay as 565.3 ps.

```
  Measurements for Transient Analysis
td                = 5.653498e-10 targ= 2.583580e-08 trig= 2.527045e-08
```

Figure 45: Worst Case Delay

IX.B.2 Post Layout

We measure the worst case delay for the same case as pre-layout. We get the worst case post layout delay as 405.3 ps.

```
  Measurements for Transient Analysis
td                =  4.053168e-10 targ=  5.568632e-09 trig=  5.163315e-09
```

Figure 46: Worst Case Delay

X Maximum Working Frequency

X..1 Pre Layout

To find the Maximum Frequency at which the circuit will work, we find the minimum clock period. We use the formula $T_{\text{clk}} >= t_{\text{pd}} + t_{\text{cq}} + t_{\text{su}}$.
From our previous analysis, we know that:
$t_{\text{pd}} = 565.3ps$
$t_{\text{cq}} = 91ps$
$t_{\text{su}} = 70.87ps$

Therefore, Minimum value of T_{clk}=565.3+91+70.87 = 727.17 ps. Therefore, the maximum clock frequency is 1/727.17=1.375 GHz.

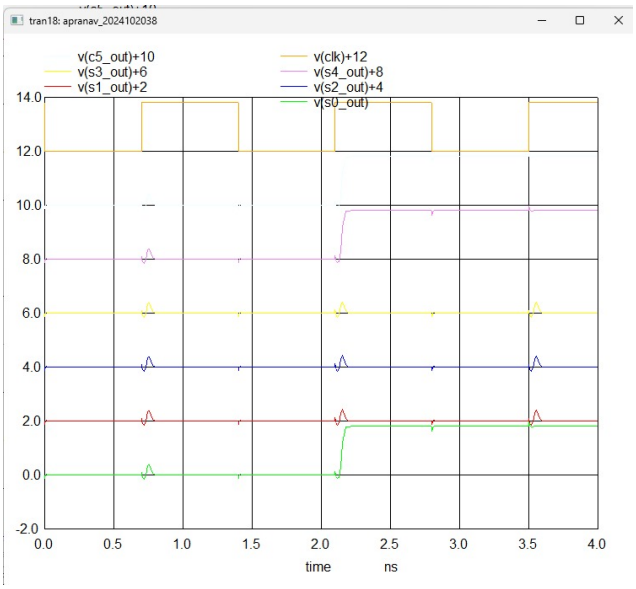

Figure 47: Maximum Working Frequency

We can observe that the circuit works and gives the correct output at this max frequency.

X.A Post Layout

$T_{\text{clk}} >= t_{\text{pd}} + t_{\text{cq}} + t_{\text{su}}$.
From our previous analysis, we know that:
$t_{\text{pd}} = 405.3ps$
$t_{\text{cq}} = 58ps$
$t_{\text{su}} = 48.6ps$

Therefore, Minimum value of T_{clk}=405.3+58+48.6 = 511.9 ps. Therefore, the maximum clock frequency is 1/511.9=1.95 GHz.

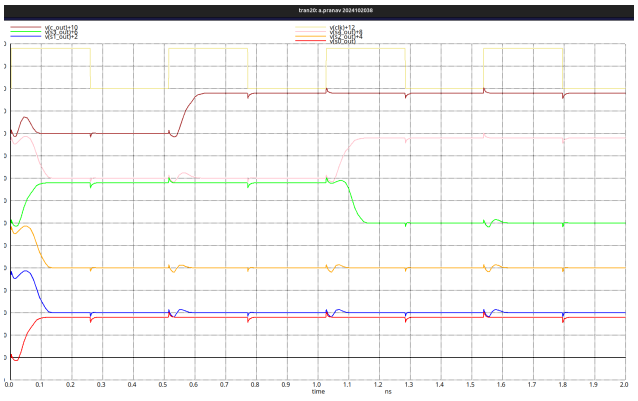

Figure 48: Simulation at Max Frequency

We can observe that the circuit works and gives the correct output at this max frequency.

XI Comparison between pre-layout and post-layout

Parameter	Pre layout	Post layout
$t_{pd,max}$	565.3 ps	405.3 ps
f_{max}	1.375 GHz	1.95 GHz

Table 5: Pre layout vs Post layout

XII Floor Plan

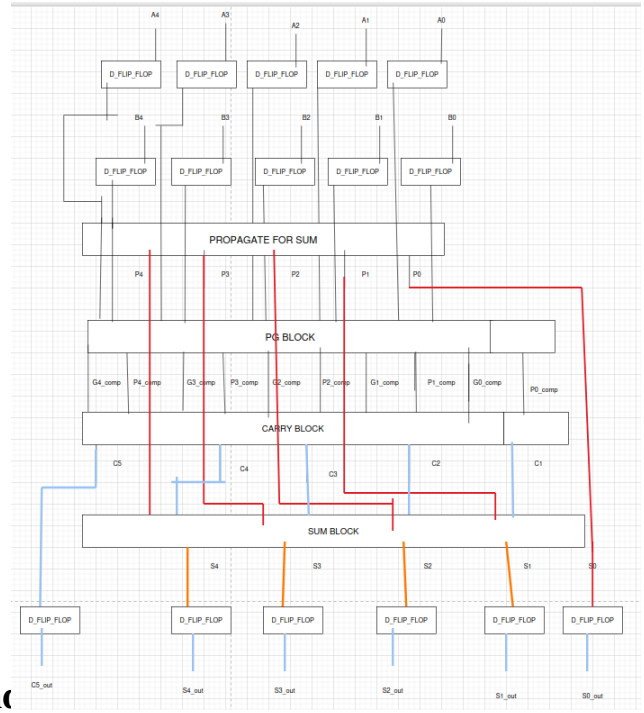

Figure 49: Floor Plan

XIII HDL Implementation

XIII.A GTKWave Waveform

Verilog Code was written to simulate the behaviour of the circuit. The following waveform was obtained for the same input case given earlier.

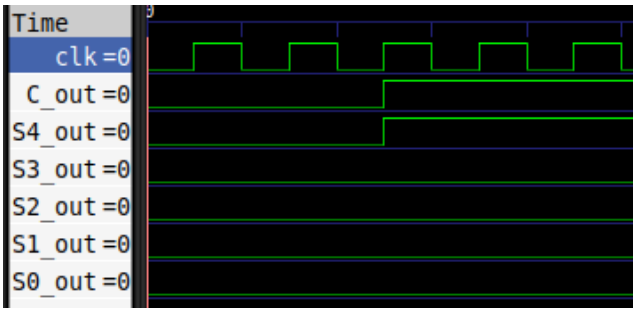

Figure 50: Gtkwave result

The waveform matches the expected output.

XIII.B FPGA implementation

To implement of FPGA, I have taken the following input test case:
A=10110
B=01111

The expected result is 100101.
The following is the result obtained:

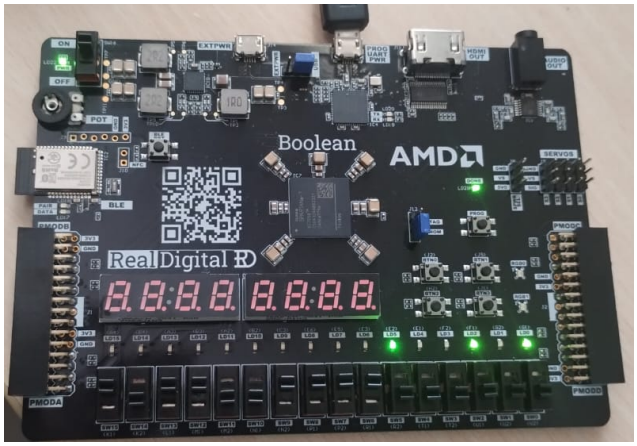

Figure 51: FPGA Result

The FPGA displays the correct output.

This output was measured using an oscilloscope. The following are the readings obtained:

The bit S0 shows a HIGH value (3.4 V).

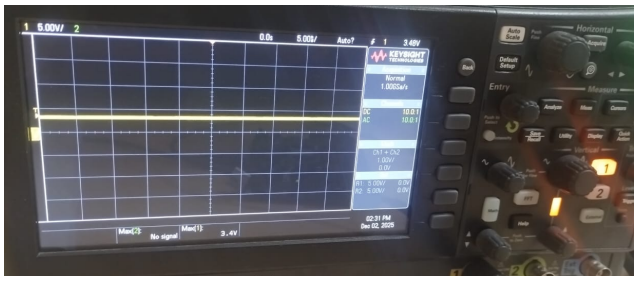

Figure 52: S0

The bit S1 shows a LOW value (200mV).

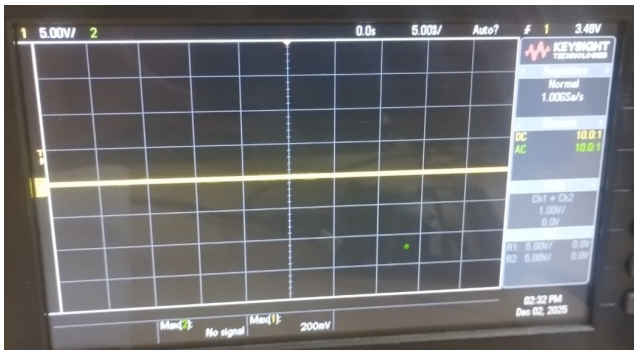

Figure 53: S1

The bit S2 shows a HIGH value (3.4 V).

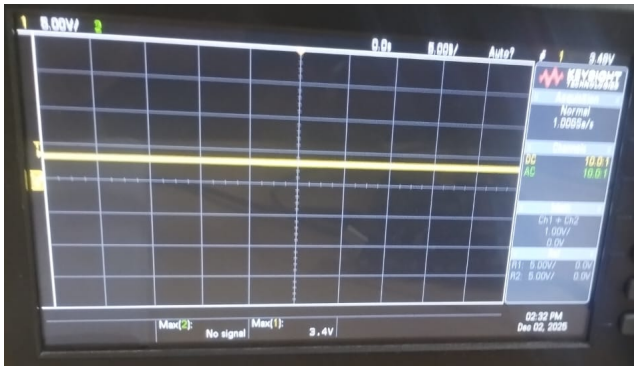

Figure 54: S2

The bit S3 shows a LOW value (200 mV).

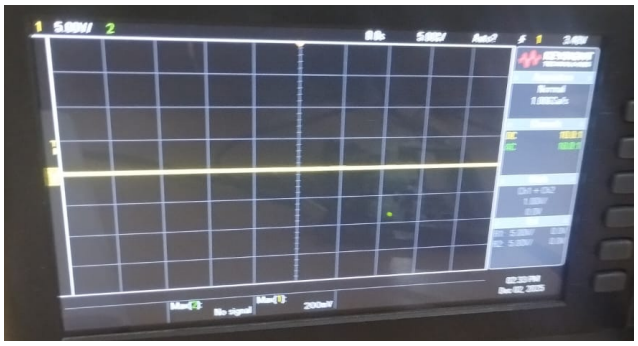

Figure 55: S3

The bit S4 shows a LOW value (0 V).

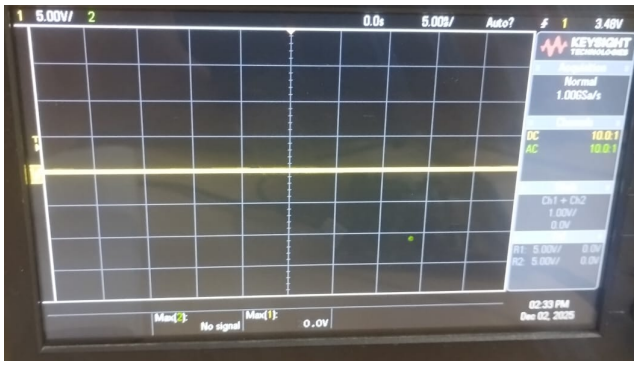

Figure 56: S4

The carry out bit C5 shows a HIGH value(3.4 V)

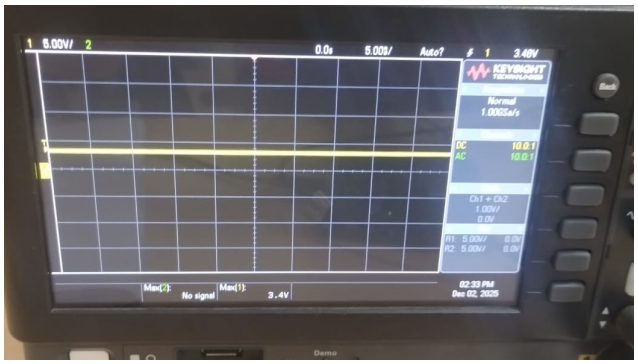

Figure 57: C5

Hence the output is 100101, which is correct.

XIV Conclusion

The 5-bit CLA was successfully designed.It was laid out in MAGIC, its netlist was written in NGSPICE. Both Pre Layout and Post Layout simulations were performed and the working of the circuit was verified. The behaviour of the circuit was modelled using VERILOG and the same was observed on the FPGA and Oscilloscope.

Acknowledgment

I would like to express my sincere gratitude to my course instructor,Dr. Abhishek Srivastava, for his valuable guidance and for providing the opportunity to work on this project.

I am also grateful to the Teaching Assistants for their constant support and help with the simulation tools. Their feedback was essential in refining the design and obtaining accurate results.

Finally, I would like to thank my **peers** for the helpful discussions and collaborative learning environment which contributed significantly to the completion of this project.

References

[1] J. Miao and S. Li, "A novel implementation of 4-bit carry look-ahead adder," *2017 International Conference on Electron Devices and Solid-State Circuits (EDSSC)*, Hsinchu, Taiwan, 2017, pp. 1-2, doi: 10.1109/EDSSC.2017.8126457.

[2] J. M. Rabaey, A. P. Chandrakasan, and B. Nikolic, *Digital Integrated Circuits: A Design Perspective*, 2nd ed., Upper Saddle River, NJ: Prentice Hall, 2003.

[3] M. M. Mano, *Digital Logic and Computer Design*, Englewood Cliffs, NJ: Prentice Hall, 1979.